djoseph-png / **py-split-integer**

Code Pull requests Actions Projects Wiki Security Insights Settings

0 stars 1.6k forks 0 watching Branches Activity
Tags

Public repository · Forked from mate-academy/py-split-integer

1 Branch 0 Tags Go to file t Go to file Add file Code

This branch is up to date with `mate-academy/py-split-integer:master` . Contribute Sync fork

bvsn Merge pull request mate-academy#1438 from Serhii-Leonenko/add_more_co...

677c1fa · 6 months ago

.github/workflows	Update test.yml	3 years ago
app	annotations added	3 years ago
tests	cecklist and pass test	3 years ago
.flake8	Update .flake8	3 years ago
.gitignore	Initial commit	3 years ago
.reviewrelatedfiles	added blank line	6 months ago
README.md	cecklist and pass test	3 years ago
checklist.md	changed to unnecessary check	3 years ago
requirements.txt	Update requirements.txt	3 years ago

README

Split integer

Read the guideline before start.

Write tests for `split_integer` function that takes 2 positive integers `value` and `number_of_parts` and returns an array containing exactly `number_of_parts` integer elements:

- the difference between the max and min number in the array should be <= 1;
- the array should be sorted ascending (from lowest to highest).

Please note: you have to use `pytest` for writing tests.

You don't need to validate arguments (they are always valid).

Examples:

```
split_integer(8, 1) == [8]
split_integer(6, 2) == [3, 3]
split_integer(17, 4) == [4, 4, 4, 5]
split_integer(32, 6) == [5, 5, 5, 5, 6, 6]
```

Notes:

- write tests in the `/app/test_split_integer.py` module;
- their names indicate what exactly they should test.

Run `pytest app/` to check if function pass your tests.

Run `pytest --numprocesses=auto tests/` to check if your tests cover all boundary conditions and pass task tests.

Note: Check your code using this checklist before pushing your solution.

Releases

No releases published
Create a new release

Packages

No packages published
Publish your first package

Languages

- **Python** 100.0%

Suggested workflows

Based on your tech stack

Python application

Create and test a Python application.

Configure

Python package

Create and test a Python package on multiple Python versions.

Configure



Pylint

Lint a Python application with pylint.

Configure

More workflows

Dismiss suggestions